



دانشکده مهندسی کامپیوتر

درس هوش مصنوعی - ترم ۱۴۰۴۲

استاد: دکتر احسان تن قطاری

پروژه شماره ۲

طراحی، پیاده‌سازی و ارزیابی جامع حملات و دفاع‌های خصمانه روی شبکه‌های عصبی عمیق

شماره گروه: ۱

آروین بقال اصل - شماره دانشجویی: ۴۰۳۱۰۵۷۹۳

آرش اکبری - شماره دانشجویی: ۴۰۳۱۰۵۷۱۴

۱ مقدمه و کلیات پروژه

در سال‌های اخیر، شبکه‌های عصبی عمیق (DNNs) به پیشرفت‌های شگرفی در حوزه‌های مختلف از جمله بینایی ماشین دست یافته‌اند، با این حال، آسیب‌پذیری این مدل‌ها در برابر دستکاری‌های جزئی و هوشمندانه ورودی—که تحت عنوان «حملات خصمانه» (Adversarial Attacks) شناخته می‌شوند—به یکی از چالش‌های اصلی در به‌کارگیری ایمن آن‌ها در سامانه‌های حساس تبدیل شده است. در این پروژه، با هدف بررسی عمیق ساختار این پدیده، به طراحی، پیاده‌سازی و ارزیابی جامع طیف متنوعی از حملات خصمانه و راهکارهای دفاعی بر روی معماری ResNet-20 پرداخته‌ایم.

در بخش نخست این مطالعه، شدت تخریب چهار الگوریتم شناخته‌شده شامل حمله FGSM، حمله PGD، حمله DeepFool و حمله Carlini & Wagner (L_2) مورد سنجش قرار گرفته است. در ادامه و به منظور مقابله با این تهدیدات، سه استراتژی دفاعی بنیادین پیاده‌سازی و مقایسه شده‌اند: «آموزش خصمانه» (Adversarial Training)، «هموارسازی تصادفی» (Randomized Smoothing) به عنوان یک روش مبتنی بر تضمین‌های آمار و احتمال، و «پالایش انتشاری» (Dif-fusion Purification) به عنوان راهکاری نوآورانه مبتنی بر مدل‌های مولد. این ارزیابی‌های جامع، تفاوت‌های ساختاری، نقاط قوت و میزان پایداری هر یک از این رویکردها را در مواجهه با طیف گوناگون حملات آشکار می‌سازند.

۲ راه‌اندازی مجموعه داده و مدل پایه

در این بخش، جزئیات مربوط به پیش‌پردازش مجموعه داده CIFAR-10، استخراج آماره‌ها، تعریف لایه نرمال‌سازی و در نهایت کپسوله‌سازی مدل پایه ResNet-20 آورده شده است.

۱.۲ بارگذاری و نمایش برخی از داده‌ها

جهت رعایت الزامات فنی حملات خصمانه، تبدیل ورودی‌ها صرفاً شامل تبدیل تصاویر به تانسورهای پایتورچ با استفاده از `transforms.ToTensor()` است. این تبدیل، مقادیر پیکسل‌ها را از بازه $[0, 255]$ به بازه اعشاری $[0, 1]$ نگاشت می‌دهد و آرایش ابعاد تصویر را از (H, W, C) به فرمت استاندارد پایتورچ یعنی (C, H, W) تغییر می‌دهد.

مجموعه داده کامل شامل ۶۰,۰۰۰ تصویر رنگی در ۱۰ کلاس تفکیک‌شده است (۵۰,۰۰۰ داده آموزش و ۱۰,۰۰۰ داده تست). برای درک بهتر داده‌ها، یک دسته (Batch) شامل ۱۰ تصویر تصادفی از داده‌های آموزش استخراج و در زیر نمایش داده شده‌اند (شکل ۱). برای نمایش درست با کتابخانه matplotlib، ابعاد تصاویر با استفاده از دستور `permute(1, 2)` به ساختار اولیه (H, W, C) بازگردانده شدند.



شکل ۱: نمونه‌ای از ۱۰ تصویر تصادفی از کلاس‌های مختلف مجموعه داده CIFAR-10

۲.۲ محاسبه آماره‌ها و پیاده‌سازی لایه نرمالایز

مدل پایه ResNet-20 نیازمند ورودی‌های نرمال‌شده با میانگین (μ) و انحراف معیار (σ) کانال‌های RGB مجموعه داده CIFAR-10 است. بدین منظور، ابتدا آماره‌های کل داده‌های آموزش محاسبه شدند:

$$\mu_{\text{RGB}} = [0.4914, 0.4822, 0.4465], \quad \sigma_{\text{RGB}} = [0.2023, 0.1994, 0.2010]$$

برای اعمال خودکار این نرمال‌سازی بر روی تصاویر خام $[0, 1]$ ، کلاس `Normalize` به عنوان یک ماژول PyTorch پیاده‌سازی شد. در ساختار این کلاس، متغیرهای μ و σ با استفاده از متد `register_buffer` تعریف گردیدند.

۳.۲ کپسوله‌سازی و آماده‌سازی مدل برای ارزیابی

در گام نهایی، لایه نرمال‌سازی در ابتدای مدل ResNet-20 قرار گرفت و در قالب یک ساختار لایه‌ای (`nn.Sequential`) کپسوله‌سازی شد:

```
model = nn.Sequential(normalize_layer, resnet20_base)
```

سپس مدل کپسوله‌شده به پردازنده گرافیکی منتقل شد و با فراخوانی متد `model.eval()` در حالت ارزیابی قرار گرفت. این دستور باعث می‌شود لایه‌هایی نظیر Batch Normalization و Dropout در حالت رفتار ثابت قرار گیرند تا فرایند محاسبه گرادیان در طول حملات خصمانه بدون تغییر در آماره‌های شبکه انجام شود.

۳ حملات خصمانه (Adversarial Attacks)

در این بخش، به بررسی و ارزیابی انواع الگوریتم‌های تولید نمونه‌های خصمانه در سناریوی جعبه سفید (White-box) می‌پردازیم. در تمامی این حملات، فرض بر این است که مهاجم دسترسی کامل به معماری شبکه، پارامترها و گرادیان‌های مدل دارد.

۱.۳ حمله FGSM (روش علامت گرادیان سریع)

حمله Fast Gradient Sign Method (FGSM) یکی از ساده‌ترین الگوریتم‌های تولید نمونه‌های خصمانه است. این روش یک حمله یک‌گامه (Single-step) محسوب می‌شود و به دلیل سرعت بالا، کاربرد گسترده‌ای در سنجش اولیه مقاومت مدل‌ها دارد.

۱.۱.۳ فرمول‌بندی ریاضی

در فرآیند آموزش استاندارد شبکه، الگوریتم بهینه‌سازی با حرکت در خلاف جهت گرادیان نسبت به وزن‌های مدل، تابع زیان را کمینه می‌کند. در مقابل، حمله FGSM با ثابت فرض کردن وزن‌های مدل (θ) ، جهت گرادیان تابع زیان (L) را نسبت به (x) محاسبه می‌کند. سپس با برداشتن یک گام در جهت بیشترین افزایش خطا، تصویر خصمانه (x_{adv}) را تولید می‌نماید:

$$x_{adv} = \text{clip}_{[0,1]}(x + \epsilon \cdot \text{sign}(\nabla_x L(\theta, x, y)))$$

در این رابطه:

- x تصویر اولیه خام در بازه $[0, 1]$ است.
- y برچسب واقعی تصویر است.
- $\nabla_x L(\theta, x, y)$ گرادیان تابع زیان (آنتروپی متقاطع) نسبت به تصویر ورودی است.
- $\text{sign}(\cdot)$ تابع علامت است که جابه‌جایی پیکسل‌ها را به مقادیر $+1$ یا -1 محدود می‌کند تا شدت تغییرات یکنواخت باشد.
- ϵ بودجه نویز خصمانه است که حداکثر میزان تغییر مجاز برای هر پیکسل را مشخص می‌کند.
- $\text{clip}_{[0,1]}$ تابع برش است که تضمین می‌کند تصویر خصمانه نهایی همچنان در بازه معتبر پیکسل‌ها یعنی $[0, 1]$ باقی بماند.

۲.۱.۳ جزئیات پیاده‌سازی و ساختار کد

توابع مربوط به این حمله شامل دو بخش اصلی هستند:

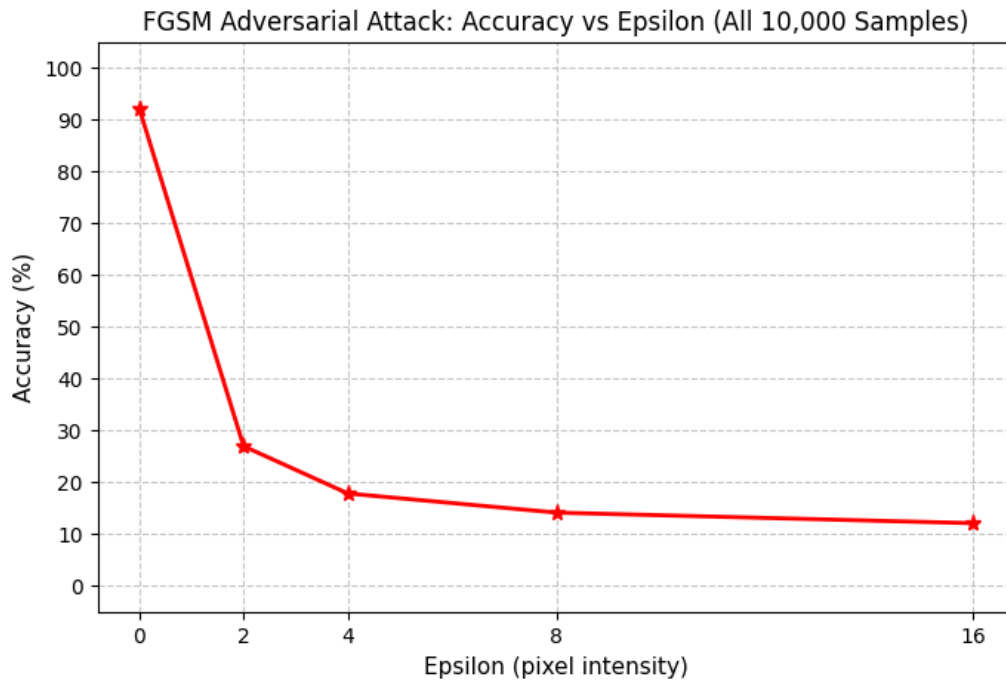
۱. تابع `fgsm_attack`: محاسبه جهت علامت گرادیان، اعمال ضریب ϵ روی تصویر و برش مقادیر به بازه $[0, 1]$.
۲. تابع `test_fgsm`: ارزیابی مدل روی داده‌های آزمایشی. در این تابع، ویژگی `requires_grad = True` روی تصویر ورودی فعال شده و پس از محاسبه پاس رفت (Forward) و تابع زیان، پاس برگشت (Backward) برای استخراج گرادیان تصویر اجرا می‌گردد.

۳.۱.۳ نتایج تجربی و تحلیل افت دقت مدل

ارزیابی حمله FGSM بر روی تمامی $10,000$ تصویر بخش تست مجموعه داده CIFAR-10 در مقادیر مختلف بودجه نویز $\epsilon \in \{0, 2/255, 4/255, 8/255, 16/255\}$ انجام گردید. جدول ۱ و شکل ۲ روند افت دقت مدل پایه ResNet-20 را با افزایش شدت نویز نشان می‌دهند.

جدول ۱: تغییرات دقت مدل ResNet-20 در برابر حمله FGSM با مقادیر مختلف ϵ

بودجه نویز (ϵ)	دقت مدل (%)
0	92.12%
2/255	26.95%
4/255	17.82%
8/255	14.13%
16/255	12.08%



شکل ۲: نمودار دقت مدل بر حسب بودجه نویز (ϵ) در حمله FGSM

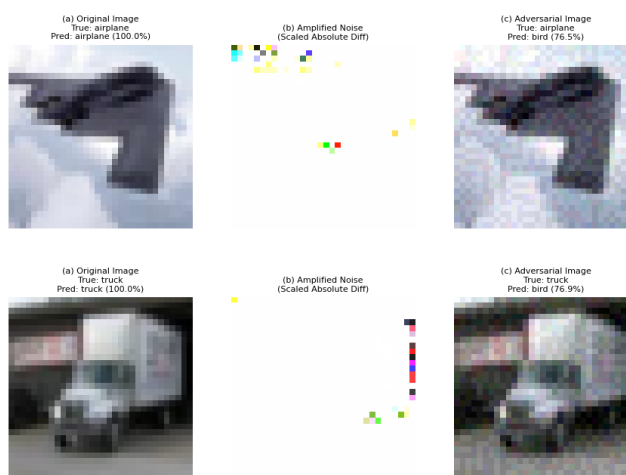
تحلیل و نتیجه‌گیری از عملکرد FGSM: تحلیل رفتار نمودار افت دقت، دو نکته کلیدی زیر را برای ما روشن می‌کند:

۱. خطی بودن محلی مرز تصمیم‌گیری (Local Linearity): با اعمال کوچک‌ترین بودجه نویز ($\epsilon = 2/255$)، دقت مدل با افت شدید 65% مواجه می‌شود! این امر نشان می‌دهد که مرز تصمیم‌گیری شبکه‌های عصبی در همسایگی نزدیک ورودی، رفتاری بسیار خطی دارد، به‌طوری که یک گام تک‌مرحله‌ای در جهت گرادیان محلی برای جابه‌جایی نمونه به آن سوی مرز کافی است.

۲. پدیده Saturation ناشی از رفتار غیرخطی کلی: با افزایش ϵ از $8/255$ به $16/255$ ، شیب افت دقت کند شده و روی محدوده 12% اشباع می‌شود. علت این پدیده آن است که مرز تصمیم‌گیری در فواصل دورتر، غیرخطی و دارای انحناست. الگوریتم تک‌گام FGSM به دلیل محاسبه یک‌باره گرادیان در نقطه اولیه، توانایی دنبال کردن انحنای پیچیده مرز را در جابه‌جایی‌های بزرگ ندارد (نیازمند حملات تکراری است).

۴.۱.۳ بررسی دیداری نمونه‌های خصمانه تولیدشده

جهت بررسی دیداری تأثیر حمله FGSM، دو نمونه از تصاویر آزمایشی که مدل پایه پیش از حمله آن‌ها را با اطمینان 100% درست تشخیص داده بود، تحت حمله با بودجه نویز $\epsilon = 8/255$ قرار گرفتند. شکل ۳ روند تخریب و نتایج خروجی را به همراه نویز بزرگ‌نمایی شده نشان می‌دهد.



شکل ۳: نمونه‌هایی از نمونه‌های خصمانه موفق در حمله FGSM با $\epsilon = 8/255$. (چپ) تصویر اصلی با پیش‌بینی صحیح، (وسط) الگوی نویز خصمانه بزرگ‌نمایی شده، (راست) تصویر خصمانه نهایی و تغییر کلاس با اطمینان بالا.

تحلیل دیداری خروجی‌ها: همان‌طور که در ستون‌های (a) و (c) دیده می‌شود، تصویر خصمانه نهایی از دیدگاه بینایی انسان کاملاً با تصویر اصلی یکسان و بدون تغییر کیفیت به نظر می‌رسد. این در حالی است که نویز ساختاریافته‌ی نشان‌داده‌شده در ستون (b)، با هدف قرار دادن بردار گرادیان، فضای ویژگی‌های عمیق شبکه را کاملاً گمراه کرده است.

۲.۳ حمله PGD (گرادیان کاهشی تصویری)

حمله PGD (Projected Gradient Descent) نسخه تکراری (Iterative) و بسیار قوی تر از FGSM است.

۱.۲.۳ فرمول بندی ریاضی

الگوریتم PGD شامل سه مرحله اصلی است:

۱. مقداردهی اولیه تصادفی (Random Start): برای رهایی از نقاط زینی (Saddle Points) محلی و شکستن تقارن در همسایگی ورودی، نقطه شروع x^0 با افزودن یک نویز یکنواخت تصادفی در بازه $[-\epsilon, \epsilon]$ شکل می گیرد:

$$x^0 = \text{clip}_{[0,1]}(x + \delta_0), \quad \delta_0 \sim \mathcal{U}(-\epsilon, \epsilon)$$

۲. به روزرسانی صعودی گرادیان (Gradient Ascent Step): در هر گام $t \in \{0, 1, \dots, K-1\}$ ، یک گام به اندازه طول گام α در جهت مستقیم افزایش زیان برداشته می شود:

$$\tilde{x}^{t+1} = x^t + \alpha \cdot \text{sign}(\nabla_{x^t} L(\theta, x^t, y))$$

۳. تصویرسازی (Projection Operator): نقطه به دست آمده ($\tilde{x}^{t+1}$) ممکن است از گوی بودجه مجاز یا بازه دامنه تصویر $[0, 1]$ خارج شود. لذا با استفاده از اپراتور تصویرسازی Π_S ، نقطه به نزدیک ترین نقطه درون مجموعه مقید نگاشت داده می شود:

$$x^{t+1} = \Pi_{x+B_\epsilon}(\text{clip}_{[0,1]}(\tilde{x}^{t+1})) = x + \text{clip}_{[-\epsilon, \epsilon]}(\text{clip}_{[0,1]}(\tilde{x}^{t+1}) - x)$$

در این فرمول بندی، $\Pi_{x+B_\epsilon}(z)$ عملگر تصویرسازی بر روی گوی L_∞ به مرکزیت x است که انحراف هر عنصر را حداکثر به بازه $[-\epsilon, \epsilon]$ محدود می سازد.

۲.۲.۳ جزئیات پیاده سازی و ساختار کد

توابع پیاده سازی شده شامل موارد زیر می باشند:

۱. تابع `pgd_attack`: این تابع ابتدا شروع تصادفی را اعمال کرده و سپس در یک حلقه K بار (در این آزمایش $K = 20$)، گرادیان را محاسبه می کند. طول گام برابر با $\alpha = \epsilon/4$ در نظر گرفته شده است. در انتهای هر گام، با محاسبه میزان انحراف $x^{t+1} - x$ و برش آن در بازه $[-\epsilon, \epsilon]$ به وسیله `torch.clamp`، عمل تصویرسازی (Projection) انجام می گیرد.

۲. تابع `test_pgd`: ارزیابی مدل روی داده های آزمایشی تحت حمله PGD و محاسبه دقت نهایی مدل.

۳.۲.۳ نتایج تجربی و تحلیل افت دقت مدل

ارزیابی حمله PGD (با ۲۰ تکرار و طول گام $\alpha = \epsilon/4$) بر روی تمامی ۱۰,۰۰۰ تصویر بخش تست مجموعه داده CIFAR-10 در مقادیر مختلف بودجه نویز $\epsilon \in \{0, 2/255, 4/255, 8/255, 16/255\}$ انجام گردید. جدول ۲ و شکل ۴ روند تخریب کامل و سقوط آزاد دقت مدل پایه ResNet-20 را نشان می دهند.

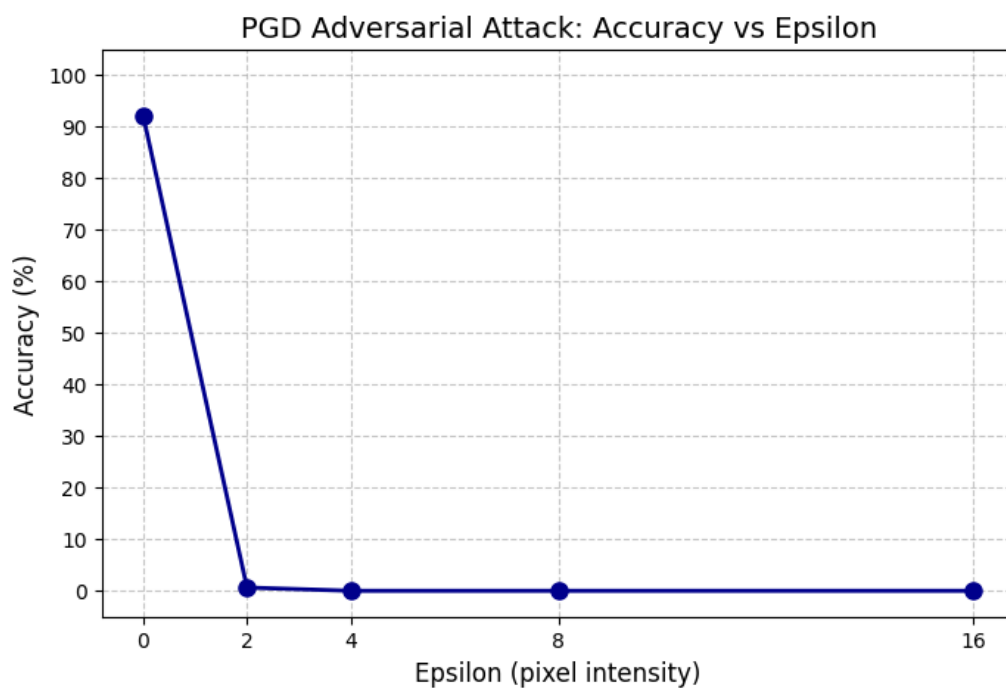
تحلیل و نتیجه گیری از عملکرد PGD: مقایسه نتایج این بخش با حمله FGSM، دو واقعیت جالب را آشکار می سازد:

۱. برتری مطلق بهینه سازی چندگامی (Iterative Optimization): برخلاف FGSM که در $\epsilon = 2/255$ دقت را به 26.95% رسانده بود، حمله PGD با تکرار ۲۰ مرحله ای و بهینه سازی پیوسته بردار نویز، دقت مدل را در همان بودجه نویز ناچیز به زیر ۱% (0.63%) و در $\epsilon \geq 4/255$ به صفر مطلق (0.00%) می رساند.

۲. شکست کامل مدل های استاندارد در برابر مهاجم جعبه سفید: این نتیجه نشان می دهد که شبکه های عصبی استاندارد به هیچ وجه در برابر بهینه سازی های مبتنی بر گرادیان مقاوم نیستند و الگوریتم PGD قادر است با کاوش دقیق در فضای ϵ -گویی، همواره نقطه ناپایداری مرز تصمیم گیری را پیدا کند.

جدول ۲: تغییرات دقت مدل ResNet-20 در برابر حمله PGD با مقادیر مختلف ϵ

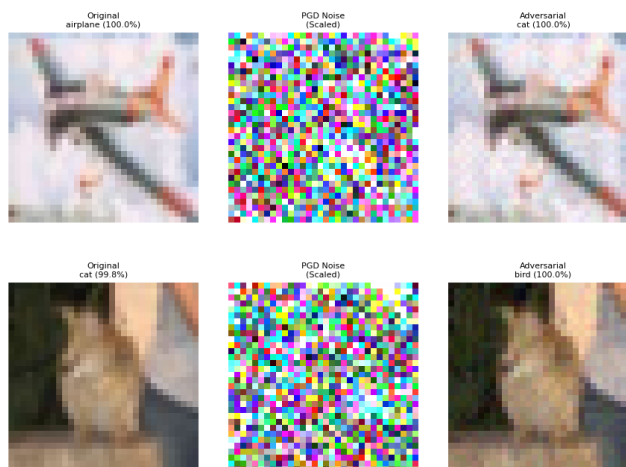
بودجه نویز (ϵ)	دقت مدل (%)
0	92.12%
2/255	0.63%
4/255	0.00%
8/255	0.00%
16/255	0.00%



شکل ۴: نمودار دقت مدل بر حسب بودجه نویز (ϵ) در حمله PGD

۴.۲.۳ بررسی دیداری نمونه‌های خصمانه تولیدشده

جهت بررسی دیداری میزان تخریب و کیفیت نویز، دو نمونه از تصاویر آزمایشی که مدل پایه آن‌ها را با اطمینان بسیار بالا درست تشخیص داده بود، تحت حمله PGD با بودجه نویز $\epsilon = 8/255$ قرار گرفتند. شکل ۵ خروجی این حمله را نشان می‌دهد.



شکل ۵: نمونه‌هایی از نمونه‌های خصمانه موفق در حمله PGD با $\epsilon = 8/255$. (چپ) تصویر اصلی، (وسط) الگوی نویز خصمانه بزرگ‌نمایی‌شده، (راست) تصویر خصمانه نهایی و تغییر کلاس با اطمینان ۱۰۰٪.

تحلیل دیداری خروجی‌ها: الگوی نویز استخراج‌شده توسط PGD (تصاویر وسط) برخلاف نویز خطی FGSM، دارای ساختار پربافت‌تر و فرکانس بالاتری است، چرا که حاصل ۲۰ مرحله اصلاح و تصویرسازی متوالی است. با این وجود، تصویر نهایی همچنان برای چشم انسان کاملاً دست‌نخورده به نظر می‌رسد.

۳.۳ حمله DeepFool

برخلاف حملات FGSM و PGD که با یک بودجه نويز ثابت (ϵ) کار می‌کنند، حمله DeepFool یک حمله مبتنی بر نُرم L_2 غیرهذفمند است که پرسش متفاوتی را مطرح می‌سازد: «کمترین اختلال ممکن که می‌تواند پیش‌بینی مدل را تغییر دهد چقدر است؟». این روش با تصویرسازی هندسی بر روی مرزهای تصمیم‌گیری، یکی از موثرترین حمله‌ها با اختلال بسیار کم به شمار می‌رود.

۱.۳.۳ فرمول‌بندی ریاضی و ایده هندسی

فرض اساسی DeepFool این است که مرز تصمیم‌گیری classifier در همسایگی ورودی به صورت خطی (ابرفضحه) قابل تقریب است.

۱. حالت دو کلاسی (ارزیابی فاصله تا ابرصفحه): برای یک طبقه‌بند خطی $f(x) = w^T x + b$ ، حداقل اختلال لازم برای عبور داده x_0 از مرز تصمیم‌گیری، تصویرسازی عمودی نقطه بر روی ابرصفحه $f(x) = 0$ است:

$$r^* = -\frac{f(x_0)}{\|w\|_2^2} w$$

۲. حالت چندکلاسی (یافتن نزدیک‌ترین مرز): در مسئله K - کلاسی، الگوریتم در هر تکرار i به دنبال کلاسی ($k \neq \hat{k}$) می‌گردد که کمترین فاصله هندسی را تا کلاس فعلی ($\hat{k}$) داشته باشد. جهت بردار اختلال (w_k) و فاصله تا مرز (f_k) به صورت زیر محاسبه می‌شوند:

$$w_k = \nabla_{x^i} f_k(x^i) - \nabla_{x^i} f_{\hat{k}}(x^i)$$

$$f_k = f_k(x^i) - f_{\hat{k}}(x^i)$$

کلاس هدف مرزی (k^*) کلاسی است که مقدار زیر را کمینه کند:

$$k^* = \arg \min_{k \neq \hat{k}} \frac{|f_k|}{\|w_k\|_2}$$

۳. گام به‌روزرسانی و پارامتر Overshoot: پس از یافتن نزدیک‌ترین مرز، بردار گام کوچک r_i محاسبه می‌شود. برای اطمینان از عبور قطعی از مرز تصمیم‌گیری (و قرار نگرفتن دقیقاً روی خود مرز)، یک ضریب (η Overshoot) به آن اضافه می‌شود:

$$r_i = \frac{|f_{k^*}|}{\|w_{k^*}\|_2^2} w_{k^*}, \quad x^{i+1} = \text{clip}_{[0,1]}(x^i + (1 + \eta)r_i)$$

حلقه تکرار فوق تا زمانی ادامه می‌یابد که پیش‌بینی مدل تغییر کند یا تعداد تکرارها از حد مجاز ($N_{\max}$) عبور نماید.

۲.۳.۳ جزئیات پیاده‌سازی و ساختار کد

کد پیاده‌سازی شده فرآیند بالا را به شیوه زیر مدیریت می‌کند:

۱. تابع deepfool_attack:

- محاسبه گرادیان کلاس اصلی: ابتدا گرادیان خروجی مربوط به کلاس پیش‌بینی شده اولیه ($\hat{k}$) نسبت به تصویر ورودی با دستور `logits[0, init_pred].backward()` استخراج می‌گردد.
- جستجو میان تمامی کلاس‌های رقیب: در یک حلقه روی $K - 1$ کلاس دیگر، گرادیان هر کلاس (g_k) استخراج شده، بردار $w_k = g_k - g_{\text{orig}}$ تشکیل یافته و فاصله $d_k = \frac{|f_k|}{\|w_k\|_2}$ ارزیابی می‌شود.
- اعمال گام و عبور از مرز: بردار اختلال بهینه r_i محاسبه شده و تصویر با ضریب $(1 + \text{overshoot})$ به‌روزرسانی و در بازه $[0, 1]$ برش داده می‌شود. پارامتر overshoot در این آزمایش برابر 0.02 تنظیم گردیده است.

۲. تابع `test_deepfool`: این تابع به دلیل ماهیت بهینه‌سازی تک به تک تصاویر در حمله `deepfool`، داده‌ها را با `batch_size = 1` پردازش می‌کند. ارزیابی بر روی ۵۰۰ نمونه صورت گرفته و به ازای هر حمله موفق، نرم L_2 بردار اختلال ($\|\delta\|_2 = \|x_{adv} - x\|_2$) محاسبه و میانگین آن به عنوان معیار ارزیابی شدت حمله گزارش می‌گردد.

۳.۳.۳ نتایج تجربی و تحلیل افت دقت مدل

همان‌طور که گفته شد، برخلاف حملاتی نظیر FGSM و PGD که نویز را بر اساس یک بودجه ثابت (ϵ) محدود می‌سازند، حمله DeepFool الگوریتمی مبتنی بر بهینه‌سازی هندسی است که هدف آن یافتن حداقل میزان نویز ممکن در نرم L_2 برای انتقال نمونه به آن سوی نزدیک‌ترین مرز تصمیم‌گیری است. ارزیابی این حمله روی ۵۰۰ نمونه آزمایشی با هایپارامترهای `max_iter = 50` و `overshoot = 0.02` انجام شد. جدول ۳ خلاصه نتایج این ارزیابی را نشان می‌دهد.

جدول ۳: نتایج ارزیابی حمله DeepFool بر روی مدل پایه ResNet-20

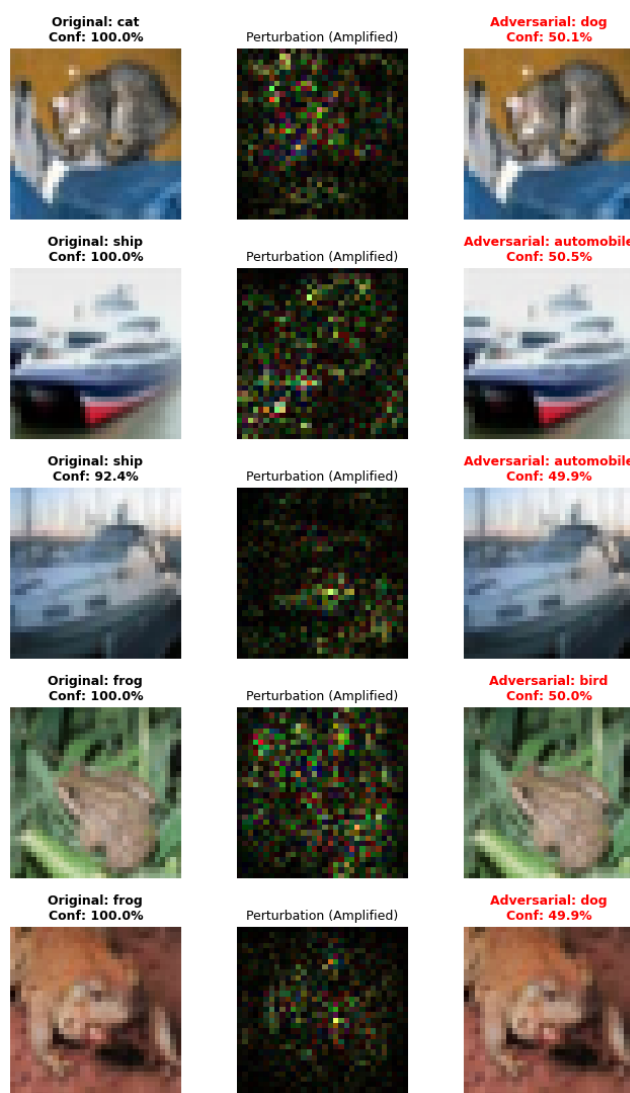
مقدار	معیار ارزیابی
500	تعداد نمونه‌های ارزیابی شده
5.00%	دقت مقاوم مدل (Robust Accuracy)
94.55%	نرخ موفقیت حمله (Attack Success Rate)
0.1179	میانگین نرم L_2 نویز (Average L_2 Perturbation)

دستیابی به نرخ موفقیت 94.55% تنها با میانگین نرم L_2 برابر با 0.1179 نشان می‌دهد که مرزهای تصمیم‌گیری شبکه ResNet-20 در فاصله‌ای بسیار نزدیک به نمونه‌های واقعی قرار دارند و مدل نسبت به کوچک‌ترین جابه‌جایی‌های خطی غیرمجاز، شدیداً آسیب‌پذیر است.

۴.۳.۳ بررسی دیداری نمونه‌های خصمانه تولیدشده

جهت بررسی هندسی خروجی‌های حمله DeepFool، نمونه‌هایی از تصاویر تغییر یافته به همراه الگوی نویز بزرگ‌نمایی شده در شکل ۶ نمایش داده شده‌اند.

تحلیل دیداری خروجی‌ها: مشاهده خروجی‌های دیداری این حمله، امضای هندسی معروف الگوریتم DeepFool را به‌خوبی نشان می‌دهد. الگوریتم DeepFool کوتاه‌ترین مسیر برداری تا نزدیک‌ترین مرز تصمیم‌گیری را محاسبه کرده و به محض اینکه احتمال کلاس خصمانه اندکی از احتمال کلاس اصلی پیشی بگیرد ($P(adv) \gtrsim P(true)$)، بهینه‌سازی را با اعمال یک ضریب `overshoot` کوچک متوقف می‌سازد. شناور بودن درصد اطمینان مدل در محدوده 50% در نمونه‌های فوق (مانند تبدیل گربه به سگ با اطمینان 50.1%) به این دلیل است که جرم احتمالی عمدتاً بین کلاس اصلی و نزدیک‌ترین کلاس رقیب جابه‌جا شده است، در نتیجه، تصویر دقیقاً در مرز مشترک مابین این دو کلاس و در کمینه‌ترین فاصله ممکن از آن قرار گرفته است.



شکل ۶: نمونه‌هایی از تصاویر خصمانه تولید شده توسط حمله DeepFool. (چپ) تصویر اصلی و اطمینان اولیه، (وسط) الگوی نویز بهینه شده L_2 ، (راست) تصویر خصمانه نهایی، کلاس جدید و اطمینان مرزی.

۴.۳ حمله Carlini & Wagner (C&W L2)

حمله Carlini & Wagner که به اختصار C&W نامیده می‌شود، پیچیده‌ترین و قدرتمندترین حمله بهینه‌سازی محور در این پروژه است. برخلاف روش‌های مبتنی بر گام‌های ساده گرادیانی (مانند FGSM و PGD)، این روش تولید تصویر خصمانه را به صورت یک مسئله بهینه‌سازی پیوسته و مقید تعریف می‌کند تا به صورت هدفمند یا غیرهدفمند، کمترین نویز ممکن در نرم L_2 را به تصویر اعمال کند.

۱.۴.۳ فرمول‌بندی ریاضی و تکنیک تغییر متغیر

هدف اصلی این حمله، حل مسئله بهینه‌سازی زیر برای یافتن اختلال δ است:

$$\min_{\delta} \|\delta\|_2^2 + c \cdot f(x + \delta)$$

در این فرمول‌بندی، ضریب $c > 0$ توازن بین «دستکاری کمینه» ($\|\delta\|_2^2$) و «موفقیت در فریب مدل» ($f(x + \delta)$) را برقرار می‌کند.

۱. تابع هدف مبتنی بر لوجیت‌ها (f): تابع زیان f بر اساس لوجیت‌های خروجی شبکه (Z) تعریف می‌شود تا بدون نیاز به لایه Softmax، تغییر کلاس صورت پذیرد:

$$f(x') = \max \left(Z(x')_y - \max_{i \neq y} Z(x')_i, -\kappa \right)$$

در این رابطه، y کلاس واقعی نمونه است و $\kappa \geq 0$ پارامتر اطمینان (Confidence) نام دارد. هرگاه $f(x') \leq 0$ گردد، یعنی لوجیت بالاترین کلاس غیر واقعی از لوجیت کلاس واقعی پیشی گرفته و حمله موفقیت‌آمیز بوده است.

۲. تغییر متغیر برای حل محدودیت مرزی $[0, 1]$: برای این‌که تصویر نهایی $x' = x + \delta$ همیشه در محدوده مجاز پیکسل‌ها ($[0, 1]$) باقی بماند، به جای بهینه‌سازی مستقیم روی δ ، بهینه‌سازی بر روی متغیر کمکی w انجام می‌گیرد:

$$x + \delta = \frac{1}{2} \left(\tanh(w) + 1 \right)$$

با این تغییر متغیر، بدون نیاز به عملگرهای برش (Clip) که مانع محاسبه گرادیان می‌شوند، تصویر حاصل تحت هر مقداری از w همواره در بازه $[0, 1]$ می‌ماند.

۲.۴.۳ جزئیات پیاده‌سازی و ساختار کد

پیاده‌سازی این حمله طبق داک پروژه برخلاف سایر روش‌ها از بهینه‌سازهای آماده یادگیری ماشین بهره می‌برد:

۱. تابع `cw_l2_attack`:

- **مقداردهی اولیه و نگاشت معکوس:** ابتدا برای جلوگیری از مقادیر بی‌نهایت در تانژانت معکوس، تصویر با `torch.clamp` در بازه $[10^{-6}, 1 - 10^{-6}]$ نگهداری شده و سپس مقدار اولیه w با فرمول $w = \text{atanh}(2x)$ (محاسبه می‌شود).
- **بهینه‌ساز Adam:** از بهینه‌ساز Adam با نرخ یادگیری $\text{lr} = 0.01$ برای به‌روزرسانی پارامترهای w استفاده می‌شود.
- **حلقه تکرار بهینه‌سازی (100 تکرار):** در هر تکرار، تصویر خصمانه x' بازسازی شده، زیان L_2 و زیان کلاس بندی f_{loss} محاسبه می‌گردند. زیان کل به صورت $\text{total_loss} = \text{l2_loss} + c * f_{\text{loss}}$ تشکیل شده و با فراخوانی `total\_loss.backward()`، گام بهینه‌سازی برداشته می‌شود.
- **ردیابی بهترین پاسخ:** تابع در طول ۱۰۰ تکرار، موفقیت حمله را رصد کرده و تصویری که هم مدل را فریب داده و هم کمترین میزان نرم L_2 را داشته، به عنوان خروجی نهایی ذخیره می‌نماید.

۲. تابع `test_cw_l2`: این تابع داده‌های آزمایشی (۵۰۰ نمونه) را با `batch_size = 1` ارزیابی کرده و نرخ دقت مقاوم (Robust Accuracy)، نرخ موفقیت حمله (Attack Success Rate) و میانگین نرم L_2 اختلالات موفق را گزارش می‌نماید.

۳.۴.۳ نتایج تجربی و تحلیل افت دقت مدل

حمله C&W L2 به عنوان قوی‌ترین و دقیق‌ترین حمله بهینه‌سازی‌محور در این پروژه، بر روی ۵۰۰ نمونه آزمایشی با هایپرپارامترهای $c = 1.0$, $\kappa = 0$, $\max_iter = 100$ و نرخ یادگیری $lr = 0.01$ ارزیابی گردید. جدول ۴ خلاصه آمار به‌دست‌آمده از این ارزیابی را نشان می‌دهد.

جدول ۴: نتایج ارزیابی حمله C&W L2 بر روی مدل پایه ResNet-20

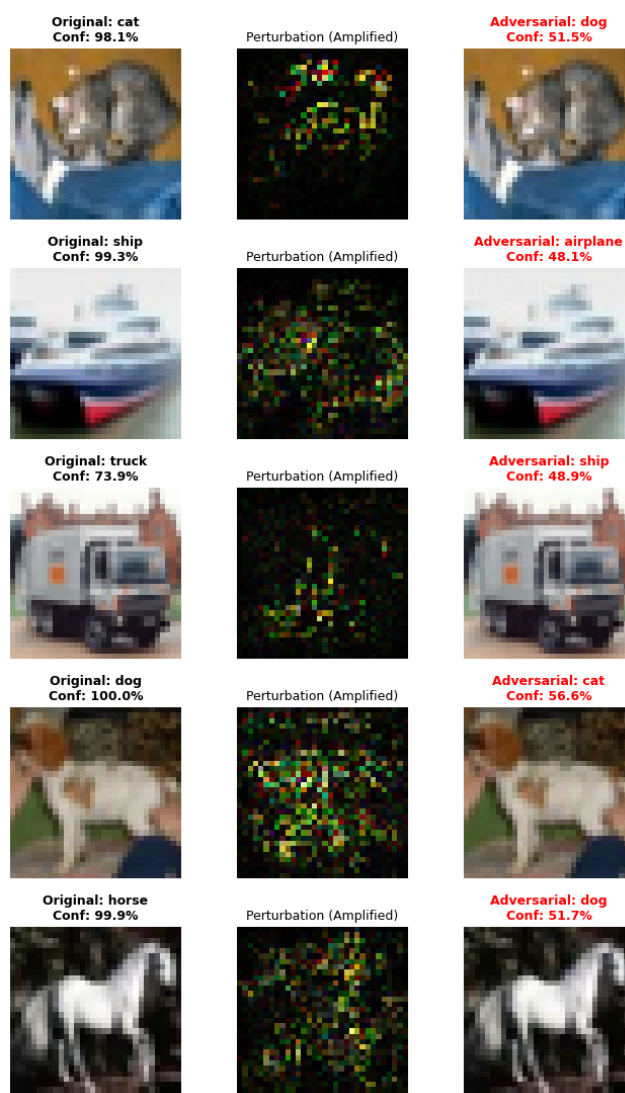
مقدار	معیار ارزیابی
500	تعداد نمونه‌های ارزیابی شده
0.00%	دقت مقاوم مدل (Robust Accuracy)
100.00%	نرخ موفقیت حمله (Attack Success Rate)
0.1466	میانگین نرم L_2 نویز (Average L_2 Perturbation)

بهینه‌سازی مستقیم تابع زیان $f(x')$ با استفاده از الگوریتم Adam و متغیر کمکی w سبب شده است که این حمله بدون استثنا بر روی تمامی ۵۰۰ نمونه موفق عمل کند و دقت مقاوم مدل را به صفر مطلق (0.00%) برساند. دستیابی به نرخ موفقیت 100% با میانگین نرم L_2 بسیار ناچیز (0.1466) نشان می‌دهد که تابع هدف ترکیبی $\text{Loss} = \|\delta\|_2^2 + c \cdot f(x')$ به بهترین شکل ممکن مرزهای تصمیم‌گیری را دور زده و مسیر بهینه‌ای برای اعمال نویز یافته است.

۴.۴.۳ بررسی دیداری نمونه‌های خصمانه تولیدشده

برای تحلیل کیفیت تصاویر خصمانه و نحوه توزیع نویز در حمله C&W L2، ۵ نمونه از خروجی‌های موفق این حمله در شکل ۷ نمایش داده شده‌اند.

تحلیل دیداری خروجی‌ها: همانگونه که در اکثر نمونه‌ها مشاهده می‌شود، میزان اطمینان مدل در پیش‌بینی جدید شناور در محدوده 50% است. این امر نشان می‌دهد به دلیل مقداردهی $\kappa = 0$ ، الگوریتم به محض برهم زدن آستانه تصمیم‌گیری، فرایند افزودن نویز را متوقف کرده است. همچنین برخلاف نویز یکنواخت در حملات L_∞ (نظیر FGSM)، الگوهای نویز در C&W (ستون وسط) بیشتر روی پیکسل‌ها و بافت‌های حساس تصویر تمرکز یافته‌اند. این امر سبب شده تا تصاویر خصمانه حاصل از نظر کیفیت دیداری و شفافیت کاملاً غیرقابل تشخیص از تصویر اصلی باقی بمانند.



شکل ۷: نمونه‌هایی از تصاویر خصمانه تولیدشده توسط حمله C&W L_2 . (چپ) تصویر اصلی و اطمینان اولیه، (وسط) الگوی نویز بهینه‌شده L_2 ، (راست) تصویر خصمانه نهایی و درصد اطمینان مرزی.

۴ پیاده‌سازی مکانیزم‌های دفاعی

پس از ارزیابی میزان آسیب‌پذیری مدل پایه در برابر انواع حملات، در این بخش به پیاده‌سازی و بررسی سه مکانیزم دفاعی برجسته جهت ارتقای دقت مقاوم (Robust Accuracy) مدل می‌پردازیم.

۱.۴ آموزش خصمانه (Adversarial Training)

آموزش خصمانه یکی از موثرترین و بنیادین‌ترین استراتژی‌های دفاعی در برابر حملات خصمانه است. ایده اصلی این روش، مقاوم‌سازی مستقیم مدل از طریق قرار دادن آن در معرض نمونه‌های سخت و اختلال‌یافته در طول فرآیند آموزش است.

۱.۱.۴ فرمول‌بندی ریاضی و مسئله Min-Max

آموزش خصمانه را می‌توان به عنوان یک مسئله بهینه‌سازی «مینیمکس» (Min-Max Optimization) فرمول‌بندی کرد:

$$\min_{\theta} \mathbb{E}_{(x,y) \sim \mathcal{D}} \left[\max_{\delta \in \mathcal{B}_{\epsilon}} L(\theta, x + \delta, y) \right]$$

این فرمول از دو لایه داخلی و خارجی تشکیل شده است:

۱. **حلقه داخلی** ($\max_{\delta}$): یافتن بدترین و خصمانه‌ترین اختلال ممکن (δ) در همسایگی $\mathcal{B}_{\epsilon}$ تصویر ورودی x که تابع زیان L را بیشینه کند (تولید نمونه خصمانه با حمله PGD).
۲. **حلقه خارجی** ($\min_{\theta}$): به‌روزرسانی پارامترهای وزن شبکه (θ) به گونه‌ای که تابع زیان روی این نمونه‌های سخت و خصمانه کمینه شود.

۲.۱.۴ جزئیات پیاده‌سازی و ساختار کد

فرآیند آموزش خصمانه در تابع `get_or_train_adv_model` به شرح زیر پیاده‌سازی شده است:

۱. **تولید نمونه‌های خصمانه بر پایه PGD**: در هر `epoch` و به ازای هر دسته داده آموزشی (Batch)، ابتدا یک حمله ۱۰ گامی PGD با نرخ گام $\alpha = 2/255$ و شعاع نویز $\epsilon = 8/255$ روی ورودی‌ها اجرا می‌شود. در طول این ۱۰ تکرار، مدل روی وضعیت `eval()` قرار می‌گیرد.
۲. **به‌روزرسانی وزن‌های شبکه**: پس از تولید داده‌های خصمانه x_{adv} ، زنجیره گرادین آن‌ها به کمک خط کد زیر `adv_data.detach()` قطع شده و داده‌های خصمانه جدید به عنوان ورودی اصلی به شبکه تغذیه می‌شوند. سپس شبکه روی وضعیت `train()` قرار گرفته و وزن‌های آن توسط بهینه‌ساز Adam با نرخ یادگیری 0.001 به‌روزرسانی می‌گردند.
۳. **مدیریت ذخیره‌سازی و ارزیابی**: این تابع ابتدا مسیر ذخیره‌سازی را بررسی کرده و در صورت وجود مدل آموزش‌دیده، آن را بارگذاری می‌کند، در غیر این صورت، مدل را به مدت ۵ epoch روی داده‌های آموزشی CIFAR-10 `finetune` شده و فایل `.pth` نهایی را ذخیره می‌سازد. تابع `evaluate_adv_on_samples` نیز میزان دقت مقاوم مدل بازآموزی‌شده را روی نمونه‌های مختلف ارزیابی می‌کند.

۲.۴ هموارسازی تصادفی (Randomized Smoothing)

هموارسازی تصادفی یکی از معدود دفاع‌های ارائه‌شده است که دارای گارانتی ریاضی اثبات‌پذیر در برابر حملات مبتنی بر نُرم L_2 می‌باشد. ایده بنیادی این روش، تبدیل یک طبقه‌بند پایه و شکننده f به یک طبقه‌بند هموارشده g است که پیش‌بینی آن بر اساس اکثریت آرای خروجی‌های مدل در همسایگی نویزی تصویر صورت می‌پذیرد.

۱.۲.۴ فرمول‌بندی ریاضی و رأی‌گیری اکثریت

برای هر تصویر ورودی x ، طبقه‌بند هموارشده $g(x)$ کلاسی را بازمی‌گرداند که بیشترین احتمال وقوع را تحت نویز گاوسی با انحراف معیار σ داشته باشد:

$$g(x) = \arg \max_c \mathbb{P}_{\epsilon \sim \mathcal{N}(0, \sigma^2 I)} (f(x + \epsilon) = c)$$

اگر احتمال کلاس برنده (p_a) در آزمون رأی‌گیری از حد مشخصی بالاتر باشد، طبق قضایای ریاضی این دفاع تضمین می‌کند که هیچ اختلال خصمانه δ با نرم L_2 کمتر از مقدار گارانتی‌شده، نمی‌تواند پیش‌بینی مدل را تغییر دهد.

۲.۲.۴ جزئیات پیاده‌سازی و ساختار کد

این دفاع در دو مرحله «آماده‌سازی مدل» و «استنتاج هموارشده» پیاده‌سازی شده است:

۱. فاین‌تیون مدل با نویز گاوسی (`get_or_train_smoothed_model`): از آنجا که شبکه‌های عصبی عادی در مواجهه با نویز شدید دچار افت شدید دقت می‌شوند، مدل پایه ابتدا به مدت ۵ اپک روی داده‌های آموزشی که با نویز گاوسی $\epsilon \sim \mathcal{N}(0, 0.25^2 I)$ ترکیب شده‌اند، فاین‌تیون می‌گردد. تصویر نویزی نهایی با `torch.clamp` در بازه $[0, 1]$ محدود می‌شود.

۲. استنتاج نویزی و رأی‌گیری (`predict_smooth`): در زمان ارزیابی، برای یک تصویر ورودی:

- تصویر اولیه $N = 100$ بار تکرار شده و به هر نسخه، یک نویز گاوسی مستقل با $\sigma = 0.25$ اضافه می‌شود.
- جهت مقیاس‌پذیری و جلوگیری از سرریز GPU، این ۱۰۰ نمونه نویزی در دسته‌های (`eval_batch_size` = 32) تایی به مدل تغذیه می‌گردند.
- با استفاده از تابع `torch.bincount`، فراوانی پیش‌بینی هر کلاس محاسبه شده و کلاسی که بالاترین تعداد رأی (p_a) را کسب کرده باشد، به عنوان خروجی قطعی دفاع انتخاب می‌شود.

۳.۴ پالایش انتشاری (Diffusion Purification)

پالایش انتشاری (DiffPure) یکی از پیشرفته‌ترین مکانیزم‌های دفاعی نوظهور است که از قدرت مدل‌های مولد انتشاری (Diffusion Models) برای پاک‌سازی تصویر پیش از ورود به طبقه‌بند استفاده می‌کند. ایده کلیدی این روش، غرق کردن اختلالات خصمانه در نویز گاوسی و سپس بازسازی تصویر تمیز اولیه از طریق مسیر معکوس انتشار است.

۱.۳.۴ فرمول‌بندی ریاضی و فرآیند دو مرحله‌ای

فرآیند پالایش انتشاری شامل دو مرحله اصلی پیش‌پردازش است:

۱. فرآیند رفت (افزایش نویز - Forward Process): در گام زمانی کوچک t ، نویز گاوسی کنترل‌شده به تصویر خصمانه x_{adv} اضافه می‌شود:

$$x_t = \sqrt{\bar{\alpha}_t} x_{adv} + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

از آنجا که ساختار نویز خصمانه بسیار ظریف و هدفمند است، اضافه کردن این نویز گاوسی باعث گم شدن و انهدام ساختار خصمانه نویز اولیه می‌شود.

۲. فرآیند بازگشت (حذف نویز - Reverse Process): با استفاده از یک شبکه U -Net پیش‌آموزش‌دیده، مسیر معکوس انتشار از گام t تا 0 طی می‌شود تا نویزهای گاوسی اضافه شده پاک‌سازی شوند و تصویر به دامنه داده‌های تمیز بازگردد. تصویر پالایش‌شده نهایی ($x_{purified}$) جهت طبقه‌بندی به مدل ResNet-20 داده می‌شود.

۲.۳.۴ جزئیات پیاده‌سازی و ساختار کد

این مکانیزم دفاعی در قالب کلاس wrapper با نام DiffPureWrapper پیاده‌سازی گردیده است:

۱. مدیریت مقیاس ورودی‌ها: از آنجا که مدل‌های انتشاری DDPM بر روی بازه $[-1, 1]$ آموزش دیده‌اند اما طبقه‌بند ResNet-20 تصاویر را در بازه $[0, 1]$ دریافت می‌کند، توابع کمکی `_to_model_range` و `_to_data_range` مسئولیت نگاشت خطی میان این دو دامنه را برعهده دارند.

۲. شتاب‌دهی استنتاج با **DDIMScheduler**: جهت کاهش بار محاسباتی سنگین مدل انتشاری، از زمان‌بند DDIM با تنظیم $N_{steps} = 20$ استفاده شده است. ابتدا با تابع `add_noise` نویز مربوط به گام $t_{purify} = 100$ اعمال شده و سپس در یک حلقه، شبکه UNet به صورت گام‌به‌گام بردار نویز را تخمین زده و تصویر پالایش شده $x_{purified}$ را بازسازی می‌نماید.

۳. ارزیابی نهایی: تصویر پالایش شده در نهایت به شبکه ResNet-20 داده شده و برچسب پیش‌بینی شده استخراج می‌گردد. به دلیل زمان‌بر بودن فرآیند استنتاج مدل‌های انتشاری، ارزیابی این دفاع بر روی زیرمجموعه محدودتری از نمونه‌ها انجام می‌شود.

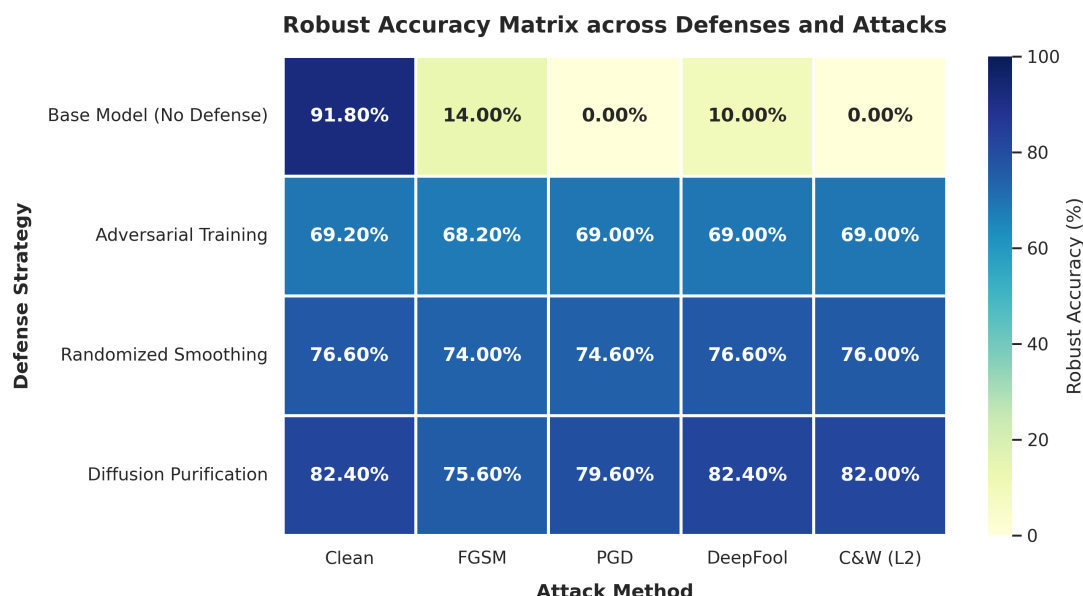
۵ ارزیابی نهایی

جهت ارزیابی جامع و مقایسه عملکرد مدل پایه و استراتژی‌های مختلف دفاعی در برابر انواع حملات خصمانه، تمامی مدل‌ها بر روی ۵۰۰ نمونه آزمایشی یکسان مورد سنجش قرار گرفتند. ماتریس ارزیابی متقابل در جدول ۵ و نمودار حرارتی (Heatmap) حاصل از آن در شکل ۸ ارائه شده است.

جدول ۵: ماتریس ارزیابی متقابل: دقت مقاوم (درصد) مدل‌ها در برابر حملات مختلف

دفاع / حمله	Clean	FGSM	PGD	DeepFool	C&W (L_2)
مدل پایه (بدون دفاع)	91.80%	14.00%	0.00%	10.00%	0.00%
Adversarial Training	69.20%	68.20%	69.00%	69.00%	69.00%
Randomized Smoothing	76.60%	74.00%	74.60%	76.60%	76.00%
Diffusion Purification	82.40%	75.60%	79.60%	82.40%	82.00%

* برای حملات FGSM و PGD بودجه نویز روی $\epsilon = 8/255$ تنظیم شده است.



شکل ۸: نمودار حرارتی (Heatmap) دقت مقاوم شبکه‌های مختلف در برابر حملات خصمانه.

تحلیل و نتیجه‌گیری جامع: همانگونه که دیده می‌شود مدل اولیه بدون دفاع، اگرچه بالاترین دقت را روی داده‌های پاک (91.80%) به ثبت رسانده است، اما در برابر حملات خصمانه به شدت شکننده عمل می‌کند، به‌طوری که دقت آن در برابر حمله ساده FGSM به 14.00% سقوط کرده و در برابر حملات بهینه‌سازی‌محور مانند PGD و C&W (L_2) کاملاً خنثی شده و به 0.00% می‌رسد.

روش Adversarial Training مقاومت کاملاً یکنواختی (حدود 68% تا 69%) در برابر تمام حملات ارائه می‌دهد. در مقابل، روش Randomized Smoothing در تمامی سناریوها سطح بالاتری از پایداری را نشان داده و دقت آن هم بر روی داده‌های پاک (76.60%) و هم تحت حملات مختلف (بین 74.00% تا 76.60%) به‌طور چشمگیری برتر از آموزش خصمانه است.

روش Diffusion Purification برترین عملکرد را در تمام شاخص‌ها به خود اختصاص داده است. این استراتژی مبتنی بر پاک‌سازی، علاوه بر کسب بالاترین دقت روی داده‌های پاک (82.40%) در میان تمام روش‌های دفاعی، در برابر انواع حملات خصمانه نیز مقاوم‌ترین رفتار را نشان داده و به دقت مقاومی بین 75.60% تا 82.40% دست یافته است.

۶ تحلیل توازن‌ها (Trade-offs) و پاسخ به پرسش‌ها

الف) پدیده بیش‌برازش مقاوم (Robust Overfitting)

پدیده بیش‌برازش مقاوم زمانی رخ می‌دهد که دقت مقاوم مدل روی داده‌های آموزش همچنان افزایش می‌یابد، اما دقت مقاوم آن روی داده‌های تست پس از چند epoch مشخص به شدت افت می‌کند. تشخیص این پدیده از طریق پایش مداوم دقت مقاوم (Robust Accuracy) روی داده‌های Validation در حین آموزش امکان‌پذیر است، به این صورت که اگر با ادامه آموزش، زیان (Loss) مقاوم تست افزایش یابد، بیش‌برازش رخ داده است. برای جلوگیری از این پدیده، مؤثرترین روش استفاده از توقف زودهنگام (Early Stopping) بر اساس دقت مقاوم Validation Set، کاهش نرخ یادگیری در گام‌های اولیه، و بهره‌گیری از تکنیک‌های میانگین‌گیری از وزن‌ها (EMA) یا هموارسازی برجسب‌ها می‌باشد.

ب) زمان استنتاج و توجیه‌پذیری DiffPure

روش DiffPure به دلیل نیاز به اجرای مکرر شبکه U-Net در گام‌های معکوس زمان انتشار (حتی با استفاده از زمان‌بندی سریع DDIM)، زمان استنتاج را از چند میلی‌ثانیه در مدل پایه به چند صد میلی‌ثانیه تا چند ثانیه به ازای هر تصویر افزایش می‌دهد. با این حال، این کندی در کاربردهای حیاتی که امنیت در اولویت قرار دارد (مانند سیستم‌های خودران یا پزشکی) کاملاً توجیه‌پذیر است، زیرا این روش بدون نیاز به بازآموزی مدل اصلی، بالاترین دقت داده‌های پاک (82.40%) و بیشترین دقت مقاوم (82.00% در برابر C&W) را حاصل کرده و برترین عملکرد دفاعی را ارائه می‌دهد.

ج) گارانتی ریاضی و افت دقت پاک در Randomized Smoothing

روش هموارسازی تصادفی با افزودن نویز گاوسی به تصویر ورودی و اخذ رأی اکثریتی از نمونه‌گیری‌های متعدد، یک شعاع تضمین‌شده ریاضی ($\text{Certified Radius } L_2$) برای مقاومت مدل ارائه می‌دهد که تضمین می‌کند هیچ ناخالصی یا حمله‌ای در محدوده این شعاع نمی‌تواند پیش‌بینی مدل را تغییر دهد.

با این حال، این ایمنی بالا نیازمند پذیرش یک توازن میان دقت و مقاومت (Accuracy-Robustness Trade-off) است. تزریق نویز گاوسی به تصاویر تمیز، بافت‌های ظریف و ویژگی‌های فرکانس بالا را فرسوده می‌کند، از آنجا که مدل مقاوم‌شده ناچار است برای حفظ پایداری از این جزئیات شکننده صرف‌نظر کند، دقت آن روی داده‌های پاک نسبت به مدل پایه دچار افت می‌شود (76.60% در مقایسه با 91.80%).